МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №7

з дисципліни "Функціональне програмування"

НА ТЕМУ "FUNCTOOLS TA OPERATOR"

Варіант 12. Навчальні онлайн-курси

Виконав:
студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: засвоїти інструменти декларативної побудови конвеєрів обробки даних за допомогою `functools.partial` та `functools.reduce`; опанувати оптимізовані гетери та виклики методів модуля `operator` (`itemgetter`, `attrgetter`, `methodcaller`); створити поліморфний завантажувач джерел через `functools.singledispatch` з підтримкою JSON та CSV; реалізувати модель із повним відношенням лінійного порядку на базі `@total_ordering` для варіанта 12 (навчальні онлайн-курси: ранжування за рейтингом, кількістю студентів і назвою).

Теоретичні відомості

Функціональна композиція в Python реалізується через комбінування часткового застосування аргументів (`functools.partial`) та лінійного згортання (`functools.reduce`). Функція `partial` фіксує частину позиційних або іменованих параметрів базової функції, генеруючи новий викликаний об'єкт із чіткою сигнатурою. Застосування `reduce` дозволяє передавати проміжний результат послідовно крізь ланцюжок чистих функцій (патерн *threading/pipeline*) без громіздких вкладених викликів або тимчасових змінних.

Модуль `operator` надає швидкі C-оптимізовані функції для доступу до полів і виклику методів замість неефективних `lambda`-виразів: `itemgetter` витягує один або кілька елементів зі словників/кортежів, `attrgetter` дістає атрибути екземплярів, а `methodcaller` виконує методи за іменем. Механізм одинарної диспетчеризації `functools.singledispatch` дозволяє створити єдиний поліморфний інтерфейс завантаження даних на основі типу першого аргументу (рядок, `Path`, відкритий файловий потік, готова колекція). Декоратор `functools.total_ordering` автоматично генерує всі шість методів порівняння за наявності коректно реалізованих `__eq__` та одного зі строгих операторів (`__lt__`), повертаючи `NotImplemented` для непідтримуваних типів.

Хід роботи

1. Предметна модель з повним порядком (models.py): згідно з варіантом 12 створено клас `CourseRank` для курсів із полями `title`, `category`, `students`, `rating`, `price`. Порядок визначено за `rating` ↓, `students` ↓, `title` ↑ з використанням `@total_ordering` та безпечною перевіркою типу `NotImplemented`:

```
from functools import total_ordering
from typing import Any

@total_ordering
class CourseRank:
    __slots__ = ("title", "category", "students", "rating", "price")

    def __init__(self, title: str, category: str, students: int, rating: float,
price: float) -> None:
        self.title = title
        self.category = category
        self.students = students
        self.rating = rating
```

```
        self.price = price

    def __repr__(self) -> str:
        return f"CourseRank({self.title!r}, {self.category!r},
students={self.students}, rating={self.rating}, price={self.price})"

    def __eq__(self, other: Any) -> bool:
        if not isinstance(other, CourseRank):
            return NotImplemented
        return (self.rating, self.students, self.title) == (other.rating,
other.students, other.title)

    def __lt__(self, other: Any) -> bool:
        if not isinstance(other, CourseRank):
            return NotImplemented
        # Порядок: rating (desc), students (desc), title (asc)
        return (-self.rating, -self.students, self.title) > (-other.rating, -
other.students, other.title)
```

2. Поліморфний завантажувач джерел через singledispatch (loaders.py): реалізовано поліморфну функцію `load_courses` для завантаження списку словників курсів із JSON/CSV рядків, об'єктів `pathlib.Path`, відкритих потоків `io.TextIOBase` та вже готових структур `list[dict]`:

```
import csv, io, json
from functools import singledispatch
from pathlib import Path
from typing import Any

def _parse_raw_text(text: str, is_csv: bool = False) -> list[dict[str, Any]]:
    s = text.strip()
    if is_csv or (not s.startswith(("[", "{")) and "," in s):
        reader = csv.DictReader(io.StringIO(s))
        return [dict(row) for row in reader]
    return json.loads(s)

@singledispatch
def load_courses(src: Any) -> list[dict[str, Any]]:
    raise TypeError(f"Непідтримуваний тип джерела: {type(src)!r}")

@load_courses.register
def _(src: str) -> list[dict[str, Any]]:
    s = src.strip()
    if s.endswith(".csv"):
        return _parse_raw_text(Path(src).read_text(encoding="utf-8"), is_csv=True)
    if s.endswith(".json"):
        return _parse_raw_text(Path(src).read_text(encoding="utf-8"), is_csv=False)
    return _parse_raw_text(src)

@load_courses.register
def _(src: Path) -> list[dict[str, Any]]:
    is_csv = src.suffix.lower() == ".csv"
    return _parse_raw_text(src.read_text(encoding="utf-8"), is_csv=is_csv)

@load_courses.register
def _(src: io.TextIOBase) -> list[dict[str, Any]]:
    content = src.read()
    return _parse_raw_text(content)

@load_courses.register
def _(src: list) -> list[dict[str, Any]]:
    if src and not isinstance(src[0], dict):
        raise ValueError("Очікується список словників")
    return src
```

3. Функціональний конвеєр на reduce, partial та operator (core.py): реалізовано ядро конвеєрної обробки `pipeline` через `reduce`, часткове застосування `partial(map, fn)` і `partial(filter, pred)`, а також функцію `clean_and_rank_courses` із нормалізацією через `methodcaller` і вибіркою через `itemgetter`:

```
from functools import partial, reduce
from operator import itemgetter, methodcaller
from typing import Any, Callable, Iterable
from models import CourseRank

get_title = itemgetter("title")
get_category = itemgetter("category")
get_students = itemgetter("students")
get_rating = itemgetter("rating")
get_price = itemgetter("price")

strip = methodcaller("strip")
title_case = methodcaller("title")
lower_case = methodcaller("lower")

def pipeline(value: Any, *funcs: Callable[[Any], Any]) -> Any:
    return reduce(lambda acc, f: f(acc), funcs, value)

def mapf(fn: Callable[[Any], Any]) -> Callable[[Iterable[Any]], Any]:
    return partial(map, fn)

def filt(pred: Callable[[Any], bool]) -> Callable[[Iterable[Any]], Any]:
    return partial(filter, pred)

def normalize_course(row: dict[str, Any]) -> CourseRank:
    return CourseRank(
        title=title_case(strip(str(get_title(row)))),
        category=lower_case(strip(str(get_category(row)))),
        students=int(get_students(row)),
        rating=float(get_rating(row)),
        price=float(get_price(row)),
    )

def clean_and_rank_courses(raw_courses: Iterable[dict[str, Any]], min_rating: float =
4.5, top: int = 3) -> list[CourseRank]:
    min_rating_pred = lambda c: c.rating >= min_rating
    top_n = lambda n: lambda it: list(it)[:n]

    return pipeline(
        raw_courses,
        mapf(normalize_course),
        filt(min_rating_pred),
        list,
        partial(sorted, reverse=True),
        top_n(top),
    )
```

4. Демонстраційний сценарій main.py: створено скрипт для демонстрації конвеєра з JSON, CSV, Path, вибірки полів через `itemgetter`/`attrgetter` та перевірки `@total_ordering`:

```
from __future__ import annotations
from operator import attrgetter, itemgetter
from pathlib import Path
from core import clean_and_rank_courses, pipeline
from loaders import load_courses
from models import CourseRank
```

```
def main() -> None:
    print("=== Лабораторна робота 7: functools та operator ===")
    json_path = Path("sample_courses.json")
    courses_from_json = load_courses(json_path)
    ranked = clean_and_rank_courses(courses_from_json, min_rating=4.5, top=3)

    print(f"Top-{len(ranked)} курсів за популярністю та рейтингом:")
    for rank, c in enumerate(ranked, 1):
        print(f"  {rank}. {c.title} ({c.category}) - Рейтинг: {c.rating}, Студентів:
{c.students}, Ціна: ${c.price}")

    csv_path = Path("sample_courses.csv")
    courses_from_csv = load_courses(csv_path)
    ranked_csv = clean_and_rank_courses(courses_from_csv, min_rating=4.5, top=2)
    print(f"
Top-{len(ranked_csv)} курсів з CSV-файлу:")
    for rank, c in enumerate(ranked_csv, 1):
        print(f"  {rank}. {c.title} - {c.rating} зірок ({c.students} учнів)")

    c1 = CourseRank("Python Pro", "dev", 1500, 4.9, 50.0)
    c2 = CourseRank("Python Basic", "dev", 1200, 4.8, 30.0)
    print(f"
Порівняння total_ordering: c1 > c2 -> {c1 > c2} (c1==c2 -> {c1 == c2})")
    print(f"Вибірка через attrgetter: {attrgetter('title', 'rating')(c1)}")

if __name__ == "__main__":
    main()
```

5. Результати виконання демонстраційної програми (main.py):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab07> python main.py
=== Лабораторна робота 7: functools та operator ===
Top-3 курсів за популярністю та рейтингом:
  1. Python Advanced (programming) - Рейтинг: 4.9, Студентів: 12500, Ціна: $49.99
  2. Machine Learning A-Z (data science) - Рейтинг: 4.85, Студентів: 18200, Ціна: $89.99
  3. Web Development Bootcamp (web) - Рейтинг: 4.8, Студентів: 25400, Ціна: $39.99

Top-2 курсів з CSV-файлу:
  1. Python Advanced - 4.9 зірок (12500 учнів)
  2. Machine Learning A-Z - 4.85 зірок (18200 учнів)

Порівняння total_ordering: c1 > c2 -> True (c1==c2 -> False)
Вибірка через attrgetter: ('Python Pro', 4.9)
```

6. Результати модульного тестування та перевірок якості коду (pytest, mypy, ruff, black):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab07> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-8.3.2, pluggy-1.5.0
plugins: hypothesis-6.115.0
collected 24 items

tests/test_core.py::test_pipeline_reduce_execution PASSED                [  4%]
tests/test_core.py::test_pipeline_empty_sequence PASSED                  [  8%]
tests/test_core.py::test_partial_mapf_and_filt PASSED                    [ 12%]
tests/test_core.py::test_clean_and_rank_filtering PASSED                 [ 16%]
tests/test_core.py::test_clean_and_rank_sorting_order PASSED             [ 20%]
tests/test_loaders.py::test_load_courses_json_str PASSED                 [ 25%]
tests/test_loaders.py::test_load_courses_csv_str PASSED                  [ 29%]
tests/test_loaders.py::test_load_courses_json_file_path PASSED           [ 33%]
tests/test_loaders.py::test_load_courses_csv_file_path PASSED            [ 37%]
tests/test_loaders.py::test_load_courses_io_stream PASSED                [ 41%]
tests/test_loaders.py::test_load_courses_list_direct PASSED              [ 45%]
tests/test_loaders.py::test_load_courses_unsupported_type PASSED         [ 50%]
```

```
tests/test_models.py::test_total_ordering_equality PASSED                [ 54%]
tests/test_models.py::test_total_ordering_comparisons PASSED             [ 58%]
tests/test_models.py::test_total_ordering_sorting PASSED                 [ 62%]
tests/test_models.py::test_total_ordering_not_implemented PASSED         [ 66%]
tests/test_operators.py::test_itemgetter_single_and_multi PASSED         [ 70%]
tests/test_operators.py::test_attrgetter_usage PASSED                    [ 75%]
tests/test_operators.py::test_methodcaller_usage PASSED                  [ 79%]
tests/test_property.py::test_property_pipeline_associativity PASSED      [ 83%]
tests/test_property.py::test_property_course_total_ordering PASSED       [ 87%]
tests/test_property.py::test_property_rank_stability PASSED              [ 91%]
tests/test_benchmark.py::test_benchmark_pipeline PASSED                  [ 95%]
tests/test_benchmark.py::test_benchmark_loaders PASSED                   [100%]

============================ 24 passed in 1.10s ============================

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab07> python -m mypy .
Success: no issues found in 5 source files

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab07> python -m ruff check .
All checks passed!

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab07> python -m black --check .
All done! 5 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи було детально засвоєно принципи функціонального програмування засобами бібліотек `functools` та `operator`. Розроблено конвеєр обробки даних на основі згортки `functools.reduce` та часткового застосування `functools.partial` без використання імперативних циклів `for`. Створено поліморфний завантажувач джерел через `@singledispatch`, який однаково обробляє рядки JSON/CSV, файлові шляхи `Path`, відкриті файлові дескриптори та колекції. Реалізовано предметний клас `CourseRank` із повною підтримкою всіх шести операторів порівняння за допомогою `@total_ordering`. Ефективність, стабільність і відповідність стандартам коду підтверджено повним набором юніт- і property-тестів та суворими лінтерами mypy, ruff і black.